

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Estrutura de Dados II

Professor: MSc. Jonathan Henrique Jeremias Souza

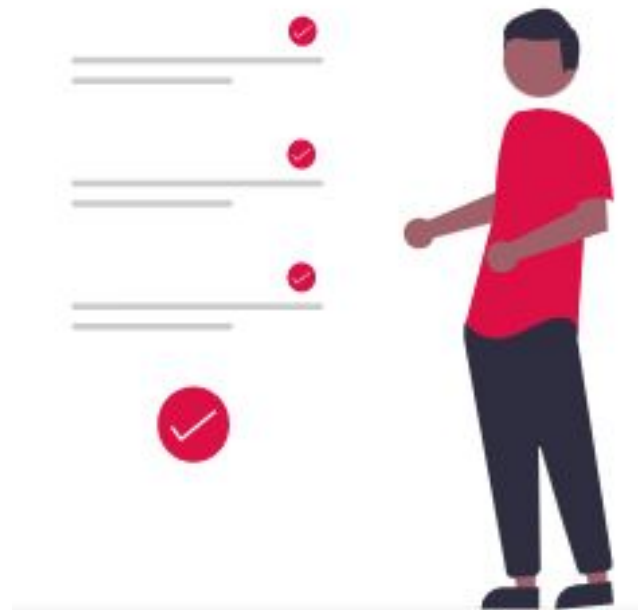
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Revisão diagnóstica;
- Tipos abstratos de dados.



MODELO DE PROGRAMAÇÃO

Tipos de dados primitivos e expressões

- Um tipo de dado é um conjunto de valores e um conjunto de operações sobre esses valores.
- Os quatro tipos de dados primitivos a seguir são a base da linguagem Java:
 - Números inteiros , com operações aritméticas (`int`)
 - Números reais , novamente com operações aritméticas (`double`)
 - Booleanos , o conjunto de valores `{true, false}` com operações lógicas (`boolean`)
 - Caracteres , os caracteres alfanuméricos e símbolos que você digita (`char`)

MODELO DE PROGRAMAÇÃO

Tipos de dados primitivos e expressões

- Um programa Java manipula variáveis nomeadas com identificadores.
- Cada variável está associada a um tipo de dado e armazena um dos valores permitidos para esse tipo de dado.
- Usamos expressões para aplicar as operações associadas a cada tipo.

term	examples	definition
<i>primitive data type</i>	int double boolean char	a set of values and a set of operations on those values (built in to the Java language)
<i>identifier</i>	a abc Ab\$ a_b ab123 lo hi	a sequence of letters, digits, _, and \$, the first of which is not a digit
<i>variable</i>	[any identifier]	names a data-type value
<i>operator</i>	+ - * /	names a data-type operation
<i>literal</i>	int 1 0 -42 double 2.0 1.0e-15 3.14 boolean true false char 'a' '+' '9' '\n'	source-code representation of a value
<i>expression</i>	int lo + (hi - lo)/2 double 1.0e-15 * t boolean lo <= hi	a literal, a variable, or a sequence of operations on literals and/or variables that produces a value

- A tabela a seguir resume o conjunto de valores e as operações mais comuns sobre esses valores para os tipos de dados `int`, `double`, `boolean` e `char` do Java.

type	set of values	operators	typical expressions	
			expression	value
<code>int</code>	integers between -2^{31} and $+2^{31}-1$ (32-bit two's complement)	<code>+</code> (add)	<code>5 + 3</code>	<code>8</code>
		<code>-</code> (subtract)	<code>5 - 3</code>	<code>2</code>
		<code>*</code> (multiply)	<code>5 * 3</code>	<code>15</code>
		<code>/</code> (divide)	<code>5 / 3</code>	<code>1</code>
		<code>%</code> (remainder)	<code>5 % 3</code>	<code>2</code>
<code>double</code>	double-precision real numbers (64-bit IEEE 754 standard)	<code>+</code> (add)	<code>3.141 - .03</code>	<code>3.111</code>
		<code>-</code> (subtract)	<code>2.0 - 2.0e-7</code>	<code>1.9999998</code>
		<code>*</code> (multiply)	<code>100 * .015</code>	<code>1.5</code>
		<code>/</code> (divide)	<code>6.02e23 / 2.0</code>	<code>3.01e23</code>
<code>boolean</code>	true or false	<code>&&</code> (and)	<code>true && false</code>	<code>false</code>
		<code> </code> (or)	<code>false true</code>	<code>true</code>
		<code>!</code> (not)	<code>!false</code>	<code>true</code>
		<code>^</code> (xor)	<code>true ^ true</code>	<code>false</code>
<code>char</code>	characters (16-bit)	[arithmetic operations, rarely used]		

MODELO DE PROGRAMAÇÃO

Expressões

- Quando uma expressão contém mais de um operador, a ordem de precedência especifica a ordem em que eles são aplicados: os operadores $*$ e $/$ (e $%$) têm precedência maior que (são aplicados antes de) os operadores $+$ e $-$;
- Entre os operadores lógicos, $!$ tem a maior precedência, seguido por $\&\&$ e depois $||$.
- Geralmente, operadores com a mesma precedência são associativos à esquerda (aplicados da esquerda para a direita). Você pode usar parênteses para alterar essas regras.

MODELO DE PROGRAMAÇÃO

Conversão de tipo

- Os números são automaticamente convertidos para um tipo mais abrangente se nenhuma informação for perdida.
- Por exemplo, na expressão $1 + 2,5$, o 1 é convertido para o valor double 1,0 e a expressão é avaliada como o valor double 3.5
- Uma conversão de tipo é uma diretiva para converter um valor de um tipo em um valor de outro tipo. Por exemplo, `(int) 3.7` é 3. Converter um double para int trunca o valor em direção a zero. (descartar a parte decimal e aproximar o número do zero.)

MODELO DE PROGRAMAÇÃO

Comparações

- Os seguintes operadores de tipo misto comparam dois valores do mesmo tipo e produzem um valor booleano:
 - igual (==)
 - diferente (!=)
 - menor que (<)
 - menor ou igual a (<=)
 - maior que (>)
 - maior ou igual a (>=)

MODELO DE PROGRAMAÇÃO

Outros tipos primitivos

- O tipo `int` em Java possui uma representação de 32 bits; o tipo `double` em Java possui uma representação de 64 bits. Java possui cinco tipos de dados primitivos adicionais:
 - Números inteiros de 64 bits, com operações aritméticas (`long`)
 - Números inteiros de 16 bits, com operações aritméticas (`short`)
 - Caracteres de 16 bits, com operações aritméticas (`char`)
 - Números inteiros de 8 bits, com operações aritméticas (`byte`)
 - Números reais de precisão simples de 32 bits, com operações aritméticas (`float`)

MODELO DE PROGRAMAÇÃO

Declarações

- Um programa Java é composto de instruções, que definem a computação criando e manipulando variáveis, atribuindo valores de tipos de dados a elas e controlando o fluxo de execução dessas operações.
- As declarações criam variáveis de um tipo específico e as nomeiam com identificadores.
- Java é uma linguagem fortemente tipada porque o compilador Java verifica a consistência. O escopo de uma variável é a parte do programa onde ela é definida.

MODELO DE PROGRAMAÇÃO

Declarações

- As atribuições associam um valor de tipo de dados (definido por uma expressão) a uma variável.
- As declarações de inicialização combinam uma declaração com uma atribuição para inicializar uma variável ao mesmo tempo em que ela é declarada.

MODELO DE PROGRAMAÇÃO

Declarações

- Atribuições implícitas. Os seguintes atalhos estão disponíveis quando nosso objetivo é modificar o valor de uma variável em relação ao valor atual:
 - Operadores de incremento/decremento: o código `i++` é uma abreviação para `i = i + 1`
 - O código `++i` é o mesmo, exceto que o valor da expressão é obtido após o incremento/decremento, e não antes.
 - Outros operadores compostos: o código `i /= 2` é uma abreviação para `i = i/2`

MODELO DE PROGRAMAÇÃO

Declarações

- As condicionais permitem uma alteração simples no fluxo de execução, as instruções são executadas em um de dois blocos, dependendo de uma condição especificada.
- Os laços (ou loops) proporcionam uma mudança mais profunda no fluxo de execução, as instruções em um bloco são executadas enquanto uma determinada condição for verdadeira.
- Nos referimos às instruções no bloco dentro de um laço como o corpo do laço.

MODELO DE PROGRAMAÇÃO

Declarações

- Interromper e continuar. O Java suporta duas instruções adicionais para uso em loops while:
 - A instrução `break`, que sai imediatamente do loop.
 - A instrução `continue`, que inicia imediatamente a próxima iteração do loop.

MODELO DE PROGRAMAÇÃO

Declarações

- Em relação à notação, muitos laços seguem este esquema:
 - inicializam uma variável de índice com algum valor e, em seguida, usam um laço while para testar uma condição de continuação envolvendo a variável de índice, onde a última instrução do laço while incrementa a variável de índice. Você pode expressar esses laços de forma compacta com for notação do Java.
- Blocos de instruções únicas. Se um bloco de instruções em uma condicional ou em um laço tiver apenas uma única instrução, as chaves podem ser omitidas. A tabela ilustra diferentes tipos de instruções Java.

statement	examples	definition
<i>declaration</i>	<code>int i;</code> <code>double c;</code>	create a variable of a specified type, named with a given identifier
<i>assignment</i>	<code>a = b + 3;</code> <code>discriminant = b*b - 4.0*c;</code>	assign a data-type value to a variable
<i>initializing declaration</i>	<code>int i = 1;</code> <code>double c = 3.141592625;</code>	declaration that also assigns an initial value
<i>implicit assignment</i>	<code>i++;</code> <code>i += 1;</code>	<code>i = i + 1;</code>

conditional (if) `if (x < 0) x = -x;`

execute a statement,
depending on boolean expression

*conditional
(if-else)* `if (x > y) max = x;
 else max = y;`

execute one or the other statement,
depending on boolean expression

loop (while)

```
int v = 0;  
while (v <= N)  
    v = 2*v;  
double t = c;  
while (Math.abs(t - c/t) > 1e-15*t)  
    t = (c/t + t) / 2.0;
```

execute statement
until boolean expression is false

loop (for)

```
for (int i = 1; i <= N; i++)  
    sum += 1.0/i;  
for (int i = 0; i <= N; i++)  
    StdOut.println(2*Math.PI*i/N);
```

compact version of while statement

MODELO DE PROGRAMAÇÃO

Matrizes

- Um array armazena uma sequência de valores que são todos do mesmo tipo.
- Se tivermos N valores, podemos usar a notação $a[i]$ para nos referirmos ao i -ésimo valor para qualquer valor de i de 0 a $N-1$.

MODELO DE PROGRAMAÇÃO

Matrizes

- Criando e inicializando um array.
- Criar um array em um programa Java envolve três etapas distintas:
 - Declare o nome e o tipo da matriz.
 - Crie a matriz.
 - Inicialize os valores da matriz.

MODELO DE PROGRAMAÇÃO

Matrizes

- Inicialização padrão de arrays
 - Para economizar código, frequentemente aproveitamos a convenção de inicialização padrão de arrays do Java e combinamos todas as três etapas em uma única instrução.
 - O valor inicial padrão é zero para tipos numéricos e falso para tipos booleanos .
- Declaração de inicialização.
 - Podemos especificar os valores de inicialização em tempo de compilação, listando os valores literais entre chaves, separados por vírgulas.

long form

```
double[] a;  declaration  
a = new double[N];  creation  
for (int i = 0; i < N; i++)  
    a[i] = 0.0;  initialization
```

short form

```
double[] a = new double[N];
```

initializing declaration

```
int[] a = { 1, 1, 2, 3, 5, 8 };
```

MODELO DE PROGRAMAÇÃO

Matrizes

- Usando um array.
 - Uma vez que criamos um array, seu tamanho é fixo.
 - Um programa pode se referir ao comprimento de um array `a[]` com o código `a.length`
 - O Java realiza verificação automática de limites, se você acessar um array com um índice inválido, seu programa será encerrado com uma exceção `ArrayIndexOutOfBoundsException` .

MODELO DE PROGRAMAÇÃO

Matrizes

- Aliasing.
 - O nome de um array refere-se ao array inteiro, se atribuirmos um nome de array a outro, ambos se referirão ao mesmo array, como ilustrado no seguinte fragmento de código.
 - Essa situação é conhecida como aliasing e pode levar a bugs sutis.

```
int[] a = new int[N];  
...  
a[i] = 1234;  
...  
int[] b = a;  
...  
b[i] = 5678;    // a[i] is now 5678.
```

MODELO DE PROGRAMAÇÃO

Matrizes

- Matrizes bidimensionais.
 - Uma matriz bidimensional em Java é uma matriz de matrizes unidimensionais.
 - Uma matriz bidimensional pode ser irregular (suas matrizes podem ter comprimentos diferentes), mas geralmente trabalhamos com matrizes bidimensionais M por N (para parâmetros M e N apropriados).
 - Para nos referirmos à entrada na linha i e coluna j de uma matriz bidimensional `a[][]`, usamos a notação `a[i][j]`.

MODELO DE PROGRAMAÇÃO

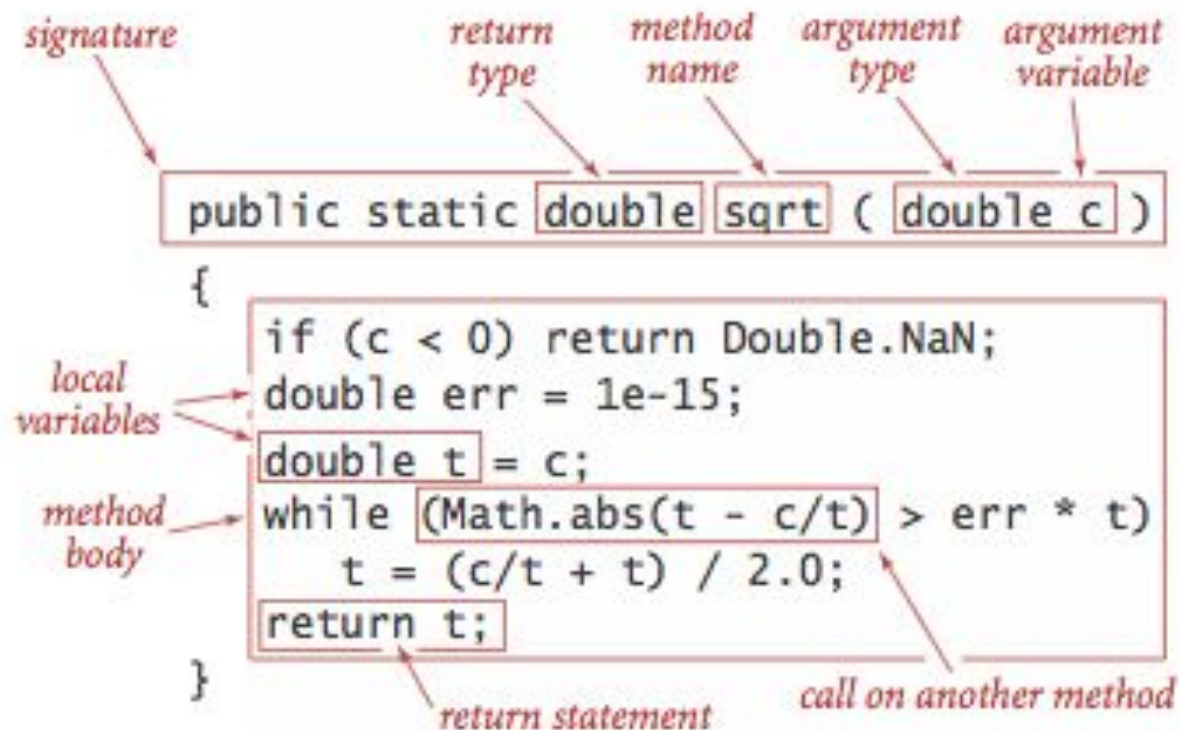
Métodos estáticos

- Em muitas linguagens de programação, os métodos estáticos são chamados de funções, pois podem se comportar como funções matemáticas.
- Cada método estático é uma sequência de instruções que são executadas, uma após a outra, quando o método estático é chamado .

MODELO DE PROGRAMAÇÃO

Métodos estáticos

- Definindo um método estático.
 - Um método encapsula uma computação definida como uma sequência de instruções.
 - Um método recebe argumentos (valores de tipos de dados específicos) e calcula um valor de retorno de algum tipo de dado ou causa um efeito colateral.
 - Cada método estático é composto por uma assinatura e um corpo .



MODELO DE PROGRAMAÇÃO

Métodos estáticos

- Invocando um método estático.
 - A chamada de um método estático consiste em seu nome seguido por expressões que especificam os valores dos argumentos entre parênteses, separados por vírgulas.
 - Quando um método é chamado, suas variáveis de argumento são inicializadas com os valores das expressões correspondentes na chamada.
 - Uma instrução ``return`` encerra um método estático, retornando o controle para quem o chamou.
 - Se o método estático deve calcular um valor, esse valor deve ser especificado em uma instrução ``return``.

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- ASCENCIO, A.F.G; ARAÚJO, G. S.; Estruturas de dados: algoritmos, análise da complexidade e implementações em JAVA e C/C++. São Paulo: Pearson, 2010
- PUGA, S.; RISSETTI, G.; Lógica de programação e estruturas de dados com aplicações em Java. 2ª Ed.; Pearson, 2009
- TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J.; Estruturas de Dados Usando C. São Paulo: Pearson, 2009